

FIT3155

SAMPLE EXAM PAPER

FIT3155 SAMPLE EXAM PAPER

Generated Sample Exam 05

INSTRUCTIONS

- This exam is worth 60 marks.

General exam technique

- Some students lose marks by not attempting all questions.
- Suppose you are likely to get 3/6 on a question after spending say 10 minutes on it. Spending another 10 minutes on the same question may gain you at most 3 additional marks. In contrast, spending those 10 minutes on a new question could earn you another 6 marks. The key point is: do not get stuck on one question.
- A good strategy is to first answer the questions you are most confident about before attempting the others. In other words, answering questions strictly serially during an exam can be suboptimal.
- Do not waste time answering an unrelated question in the hope of gaining partial marks. Marking will be based on the question that was actually asked.
- Where necessary, justify your answers with clear and concise explanations, without being overly wordy.

Question 1**(6 marks)**

Let $txt[1 \dots n]$ be a text and $pat[1 \dots m]$ a pattern, both drawn from an alphabet Σ , with $m \leq n$.

Write pseudocode for an efficient algorithm that outputs *every* starting position j in txt at which pat occurs exactly, i.e. every j such that $txt[j \dots j + m - 1] = pat[1 \dots m]$. State the worst-case time complexity of your algorithm and briefly justify it.

(For this problem, you may assume that `zalg` is available as a built-in function that takes a string as input and returns the array of its Z-values. Do not write pseudocode for `zalg`. *Hint:* consider the Z-values of a single carefully chosen concatenation, using a separator symbol that appears in neither pat nor txt .)

(Page intentionally left blank for working space)

Question 2**(6 marks)**

In the Boyer–Moore algorithm for exact matching of the pattern $pat[1 \dots m]$, the good-suffix array $gs[1 \dots m + 1]$ is used to decide a safe shift after a mismatch. The intended meaning is the one used in the shifting rule: when a right-to-left scan mismatches at position k (so the suffix $pat[k + 1 \dots m]$ has just been matched), the pattern may be shifted $m - gs[k + 1]$ positions to the right, where $gs[k + 1]$ is the right-end position in pat of the rightmost *internal* copy of the matched suffix $pat[k + 1 \dots m]$ whose immediately preceding character differs from $pat[k]$ (and $gs[k + 1] = 0$ if no such copy exists).

Write pseudocode that, given $pat[1 \dots m]$, constructs the entire array $gs[1 \dots m + 1]$. You may assume that the Z-suffix values $Zsuf[1 \dots m]$ of pat are available, where $Zsuf_i$ is the length of the longest substring of pat ending at position i that is identical to a suffix of pat (these are simply the Z-values of the reversed pattern, read back to front). State the worst-case time and space complexity of your construction.

(Page intentionally left blank for working space)

Question 3**(6 marks)**

Let $S[1 \dots n - 1]\$$ be a string terminated by a unique symbol $\$$ that is lexicographically smaller than every other character, and let $L[1 \dots n]$ be its Burrows–Wheeler Transform (the last column of the sorted matrix of cyclic rotations of S).

You are given only the transform

$$L = \text{rttaar}\$.$$

Using the last-to-first (LF) mapping, invert this BWT to recover the original string S . Show your working clearly: state the first column F that you use, explain how each successive character of S is recovered, and present the final reconstructed string.

(Page intentionally left blank for working space)

Question 4**(6=3+3 marks)**

These questions relate to the suffix tree of a string $S[1 \dots n]\$, built by Ukkonen's linear-time algorithm, where $\$$ is a unique terminal symbol.$

- (i) Prove that the completed suffix tree has exactly n leaves and at most $n - 1$ internal nodes (not counting the root). Your argument should make clear why every internal node has at least two children, and why the terminal symbol $\$$ guarantees there is one leaf per suffix.
- (ii) Define what a *suffix link* is. Then explain how suffix links are used to move from one suffix extension to the next *within a single phase* of Ukkonen's algorithm, and why their use (together with the skip/count trick) is essential to achieving the overall $O(n)$ running time.

(Page intentionally left blank for working space)

Question 5**(6=3+3 marks)**

- (i) Work out the Elias-Omega binary codeword for the positive integer 42. Show each length component of your codeword and the order in which the components are concatenated.
- (ii) Encode the string

$$S = \text{ababab}$$

using the LZ77 method, producing the sequence of tuples $[\textit{offset}, \textit{length}, \textit{next character}]$. Assume an unbounded search window and unbounded lookahead, and allow a copy whose length exceeds its offset (overlapping copy). Show how each tuple is formed.

(Page intentionally left blank for working space)

Question 6**(6=3+3 marks)**

These questions concern the number-theoretic fact at the heart of the Miller–Rabin primality test.

- (i) Prove that if p is prime, then the only solutions of

$$x^2 \equiv 1 \pmod{p}$$

are $x \equiv 1 \pmod{p}$ and $x \equiv p - 1 \pmod{p}$. (In other words, the only square roots of 1 modulo a prime are the *trivial* ones, ± 1 .)

- (ii) Write $n - 1 = 2^s t$ with t odd. Using part (i), explain why the Miller–Rabin test, on examining the sequence $a^t, a^{2t}, a^{4t}, \dots, a^{2^{s-1}t} \pmod{n}$, can correctly declare n composite. In particular, explain what makes a base a a *witness* to the compositeness of n .

(Page intentionally left blank for working space)

Question 7**(6 marks)**

Starting from an initially empty Fibonacci heap, the following keys are inserted, one at a time, in this order:

7, 3, 8, 1, 5, 9, 2, 6.

A single EXTRACT-MIN operation is then performed.

Use the following conventions: INSERT lazily adds a new single-node tree to the root list; after the minimum is removed, the consolidation step processes the root list and repeatedly links two roots of *equal* degree by making the root with the larger key a child of the root with the smaller key.

- (i) Draw the heap (i.e. the root list and the trees it contains) at the moment just *before* EXTRACT-MIN is performed, and indicate **H.min**.
- (ii) Perform the EXTRACT-MIN. Draw the resulting heap *after* consolidation, clearly showing every tree, the parent-child relationships, and the new **H.min**.

(Page intentionally left blank for working space)

Question 8**(6 marks)**

A first-in-first-out queue is implemented using two stacks, IN and OUT, each supporting PUSH and POP at unit cost per element:

- ENQUEUE(x): push x onto IN.
- DEQUEUE(): if OUT is empty, repeatedly pop every element off IN and push it onto OUT; then pop and return the top of OUT.

Both stacks are initially empty. Using the *accounting* (banker's) method of amortized analysis, show that the amortized cost of both ENQUEUE and DEQUEUE is $O(1)$. State the charge you assign to each operation, state the credit invariant your scheme maintains, and verify that the total stored credit can never become negative over any sequence of operations starting from two empty stacks.

(Page intentionally left blank for working space)

Question 9**(6=3+3 marks)**

Consider a B-tree with minimum degree t , in which every node stores between $t - 1$ and $2t - 1$ keys (the root may hold fewer). Insertion uses the top-down splitting strategy: whenever a full child (one holding $2t - 1$ keys) is encountered on the way down, it is split before being entered.

- (i) Write pseudocode for the operation $\text{SPLIT-CHILD}(x, i)$, which splits the full child y (the i -th child of node x) into two nodes and promotes its median key into x . State the worst-case time complexity of SPLIT-CHILD in terms of t .
- (ii) Explain why SPLIT-CHILD preserves all the B-tree invariants (the bounds on the number of keys per node, and the requirement that all leaves remain at the same depth), and explain why, under this insertion strategy, the height of the B-tree can increase only when the root itself is split.

(Page intentionally left blank for working space)

Question 10**(6=3+3 marks)**

- (i) Solve the following linear program using the tableau simplex algorithm. Show the tableau after each pivot, and state the optimal values of x , y , and z .

$$\begin{array}{ll}\text{Maximize} & z = 3x + 4y \\ \text{Subject to} & 2x + y \leq 10, \\ & x + 3y \leq 15, \\ & x \geq 0, \quad y \geq 0.\end{array}$$

- (ii) Explain how, while running the tableau simplex method, one can recognise from a tableau that the linear program is *unbounded* (i.e. the objective can be increased without limit). Describe precisely the condition on a chosen entering column that signals unboundedness, and state what the algorithm should report in that case.

(Page intentionally left blank for working space)

End of Exam